Minimal package

./stack.yaml

```
resolver: ghc-9.2.4

extra-deps: []

flags: {}

extra-package-dbs: []

system-ghc: false

require-stack-version: ">=2.9"
```

./package.yaml

```
name:                myproject
version:             0.1.0.0
github:              "githubuser/myproject"
license:             BSD3
author:              "Author name here"
maintainer:          "example@example.com"
copyright:           "2022 Author name here"

extra-source-files:
- README.md
- CHANGELOG.md

description:         Please see the README on GitHub at
<https://github.com/githubuser/myproject#readme>

dependencies:
- base >= 4.7 && < 5
- mtl

ghc-options:
- -Wall

library:
  source-dirs: src
executables:
  myMain:
    main:                ./app/Main.hs
    dependencies:
    - myproject

tests:
  myTest:
    main:                ./test/Spec.hs
    dependencies:
    - myproject
```

src/Lib.hs

```
module Lib ( hello ) where
hello :: IO ()
hello = putStrLn "Hello"
```

app/Main.hs

```
module Main (main) where
import Lib
main :: IO ()
main = hello
```

test/Spec.hs

```
main :: IO ()
main = error "Test suite not yet implemented"
```

```
## Usage
#
$ stack build            # Install GHC, dependencies and
compile
$ stack exe -- myMain     # Run
$ stack exec -- ghci      # Raw GHCI REPL
$ stack repl             # GHCI Replc with packages
loaded
$ stack build --test      # Compile test + run
$ stack clean            # Delete caches e.g. build
$ stack clean --full      # Delete ./.stack-work
```

Setup shell autocomplete
Ref: https://docs.haskellstack.org/en/stable/shell_autocompletion/
Load **completion file**

1) Create completion file

```
$ stack --bash-completion-script $(which stack) >
~/.config/completion/stack.bash
```

2) Add to *rc file

```
# e.g. .bashrc
#!/usr/bin/env bash
source ~/.config/completion/stack.bash
```

Remove unused packages/GHC
Ref: https://stackoverflow.com/questions/48627923/haskell-stack-

commands-for-uninstall-clean-up

Looks like there is no command.
Delete manually from the **stack root** ~/.stack
Or can just delete the whole ~/.stack

```
# Solution 1) Remove specific GHC
# e.g. Remove GHC 9.2.3
$ stack path --programs                                    #
Check where GHC is installed
/Users/yuinishizawa/.stack/programs/x86_64-osx
$ rm -rf  ~/.stack/programs/x86_64-osx/ghc-9.2.3      #
Remove

# Solution 2) Remove all cache
$ rm -rf ~/.stack
```

[CLI] Enable profiling
GHC profiling: https://downloads.haskell.org/ghc/latest/docs/users_guide/profiling.html#
Stack debugging: https://docs.haskellstack.org/en/stable/GUIDE_advanced/#debugging

Problem: need equivalent of GHC's **--prof** flag.
Solution: use Stack's **--profile** flag

```
# usage 1) profile executables
$ stack build --profile ...
$ stack exec --profile -- myprogram +RTS -p -RTS
$ cat myprogram.prof

# case 2) profile test
$ stack test --profile --test-arguments="+RTS -p
-RTS" ...
$ cat myprogram-test.prof

# case 3) profile benchmark
$ stack bench --profile --bench-arguments="+RTS -p
-RTS" ...
$ cat myprogram-bench.prof
```

[CLI] Build to output stacktrace
Ref: https://stackoverflow.com/questions/57924638/stack-run-with-profiling-in-haskell
Use **--profile** flag when running and building

e.g. suppose

```
-- main.hs
main = do
  head []
  putStrLn "Hello"
```

With package.yaml
i.e. component name = myMain

```
# package.yaml
...

executables:
  myMain:
    main:           ./main.hs
    source-dirs:    .

...
```

```
# Step 1) build with profile
$ stack --profile build

# Step 2) exec with profile
$ stack --profile exec -- myMain +RTS -xc

*** Exception (reporting due to +RTS -xc): (THUNK_1_0),
stack trace:
  GHC.List.CAF
  --> evaluated by: Main.main
test: Prelude.head: empty list
```

[CLI] Install a tool with stack on the fly
Ref: https://stackoverflow.com/questions/53036125/stack-hdevtools-is-a-library-dependency-but-the-package-provides-no-library-n
i.e. NOT a **dependency**, but just an executable tool.
i.e. equivalent to calling **cabal install**.
e.g. hp2pretty

Solution: use the **--copy-compiler-tool** flag

```
# e.g. install hp2pretty
$ stack build --copy-compiler-tool hp2pretty

# Use
$ stack exec -- hp2pretty ...
# Or enter shell
$ stack exec -- bash ...
$ hp2pretty ...
```

Usage: internal libraries
Problem
Need to use **internal only** libraries in different targets
e.g. use library in main program + tests

```
src
  main.hs
  A.hs
  Lib
    B.hs
```

```
test
  main.hs
```

In src/main.hs

```
-- src/main.hs
import A
import Lib.B

main = ... exec ...
```

In test/main.hs

```
-- test/main.hs

import A
import Lib.B

main = ... tests ...
```

Solution 1) [**DEPRECATED**] use **internal-libraries** of package.yaml
Ref: https://stackoverflow.com/questions/53890793/how-does-one-have-multiple-libraries-using-hpack
HPack: https://github.com/sol/hpack

Can just use the **inetrnal-libraries** field of package.yaml

e.g. with file structure

```
src
  main.hs            uses A, Lib.B
  A.hs
  Lib
    B.hs
test
  main.hs            uses A, Lib.B
```

package.yaml

```
# package.yaml
...

executables:
  main-exe:
   main:        main.hs
   source-dirs:  ./src                          # can use
in main because same dir

internal-libraries:
  main-lib:
   source-dirs: ./src
   exposed-modules:
```

```
   - A
   - Lib.B

test:
  main-test:
    main:           main.hs
    source-dirs:     ./test
    dependencies:
    - main-lib                              # use in
test
```

DEPRECATION:
Issues with this solution:
- inefficient build : **every** internal library gets recompiled for **every target,** even if the target doesn't use it.
Ref: https://stackoverflow.com/questions/58538238/stop-stack-install-from-building-internal-libraries-that-no-exes-depend-on
e.g.

```
stack test myproject:test:main-test     # rebuilds every
internal library, even ones not used by
target myproject:test:main-tes
```

- **LSP (haskell language server)** incompatibility: changes to internal-libraries don't appear until they are recompiled + LSP is restarted .
Internal libraries are not supported in hie.yaml
Ref: https://github.com/haskell/hie-bios#internal-libraries

Solution 2) use **symbolic links**
Just create symbolic links to files.
Solves both problems of internal-libraries (see above).

e.g.
File structure

```
src
  main.hs                     uses A, Lib.B
  A.hs
  Lib
    B.hs
test
  main.hs                     uses A, Lib.B
  Lib -> ../src/Lib           symlink
  A.hs -> ../A.hs             symlink
```

package.yaml

```
executables:
  main-exe:
   main:        main.hs
   source-dirs:  ./src

test:
  main-test:
```

```
    main:           main.hs
    source-dirs:     ./test
```

[Script] Use as script
[Stack] Script interpreter: https://docs.haskellstack.org/en/stable/scripts/

Use **stack** shebang

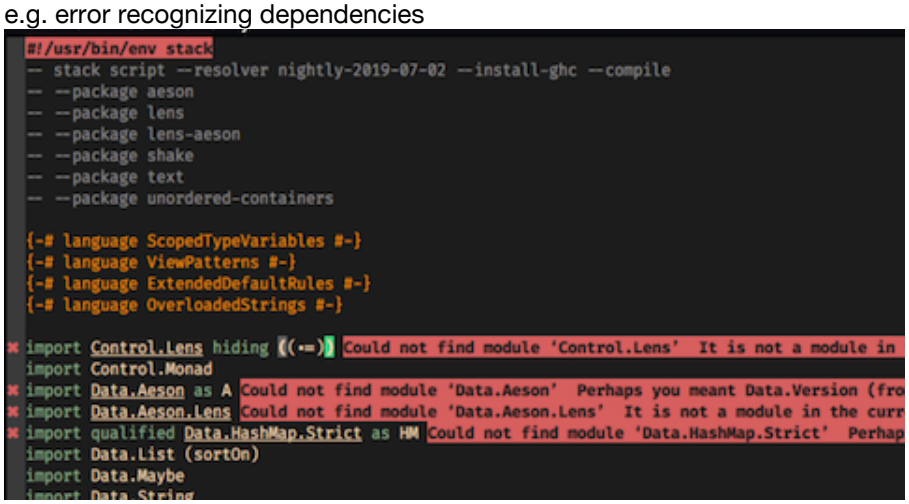
e.g. script.hs

```
#!/usr/bin/env stack
-- stack script --resolver nightly-2022-11-05 --compiler
ghc-9.2.5 —package bytestring
{-# LANGUAGE OverloadedStrings #-}
import Data.ByteString.Char8 qualified as B

main :: IO ()
main = B.putStrLn "Hello"
```

*) With HLS (Haskell Language Server)

Problem: at time of writing, HLS does not support stack scripts
[HLS] stack script workarounds: https://github.com/haskell/haskell-language-server/issues/1064
[HLS] support for stack/cabal script: https://github.com/haskell/haskell-language-server/issues/111
e.g. error recognizing dependencies



Solution: need to create package.yaml, stack.yaml, hie.yaml like a normal project

stack.yaml

```
resolver: nightly-2022-11-05
compiler: ghc-9.2.5
```

package.yaml

```
name:                Script
dependencies:
```

```
- base >= 4.14 && < 5
- bytestring >=0.10 && <0.12

ghc-options:
- -rtsopts=all
- -fprof-auto
- -fprof-auto-top
- -fprof-auto-calls
- -fprof-auto-exported
- -fprof-auto-calls
- -Wall
- -Wcompat
- -Widentities
- -Wincomplete-record-updates
- -Wincomplete-uni-patterns
- -Wmissing-export-lists
- -Wmissing-home-modules
- -Wpartial-fields
- -Wredundant-constraints
- -threaded
- -rtsopts
- -Werror               # Warning as error

default-extensions:
- Haskell2010

executables:
  script:
    main:                script.hs
    source-dirs:         .
```

hie.yaml

```
cradle:
  stack:
    component: "Script:exe:script"
```

==========
Troubleshooting

System GHC: "system-ghc: true" fails to use system GHC with error "No compiler found"

Bug
system GHC exists and is installed but Stack fails to find it
e.g.

```
$ which ghc-9.2.4
             # GHC exists
/Users/yuinishizawa/.ghcup/bin/ghc-9.2.4
```

```
$ stack build
             # ERROR
No compiler found, expected minor version match with
ghc-9.2.4 (x86_64) (based on resolver setting in /Users/
yuinishizawa/Desktop/playground/stack.yaml).
To install the correct GHC into /Users/
yuinishizawa/.stack/programs/x86_64-osx/, try running
"stack setup" or use the "--install-ghc" flag. To use
your system GHC installation, run "stack config set
system-ghc --global true", or use the "--system-ghc"
flag.
```

Problem
The system **architecture** does not match.

Solution 1
Set **arch** config in Stack
Stack arch config: https://docs.haskellstack.org/en/stable/yaml_configuration/#arch

Step 1) Check architecture used by ghcup
e.g. aarch64

```
$ ls -l ~/.ghcup/ghc/9.2.4/lib/ghc-9.2.4/lib/
total 184
drwxr-xr-x  17 yuinishizawa  staff    544 25 Dec
05:20 ./
drwxr-xr-x   4 yuinishizawa  staff    128 25 Dec
05:20 ../
drwxr-xr-x  83 yuinishizawa  staff   2656 25 Dec 05:20
aarch64-osx-ghc-9.2.4/            # aarch64
drwxr-xr-x   6 yuinishizawa  staff    192 25 Dec 05:20
bin/
-rw-r--r--   1 yuinishizawa  staff   2787 25 Dec 05:20
ghc-usage.txt
-rw-r--r--   1 yuinishizawa  staff  17943 25 Dec 05:20
ghcautoconf.h
-rw-r--r--   1 yuinishizawa  staff    837 25 Dec 05:20
ghci-usage.txt
...
```

Step 2) Set architecture in Stack config
e.g. aarch64

```
# in ~/.stack/config
system-ghc: true
install-ghc: false
arch: aarch64
# arch: x86_64        # Default
```

Step 3) Run

```
$ stack build
...
```

Solution 2
Set architecture in ghcup
ghcup config: https://www.haskell.org/ghcup/guide/#configuration

==========
Trouble shooting

[haddock] Haddock fails to build with error "haddock: internal error: ... openBinaryFile: does not exist (No such file or directory)
e.g.

```
haddock: internal error: /Users/yuinishizawa/.stack/
snapshots/aarch64-osx/
134e890a5dd4f65737d6ce4f319374a6d863bbc3ea4aed1de64071c4
72c20a89/9.2.5/doc/case-insensitive-1.2.1.0/doc-
index.json: openBinaryFile: does not
 exist (No such file or directory)
```

Solution
Not sure why the file doesn't exist, but just create an **empty file**

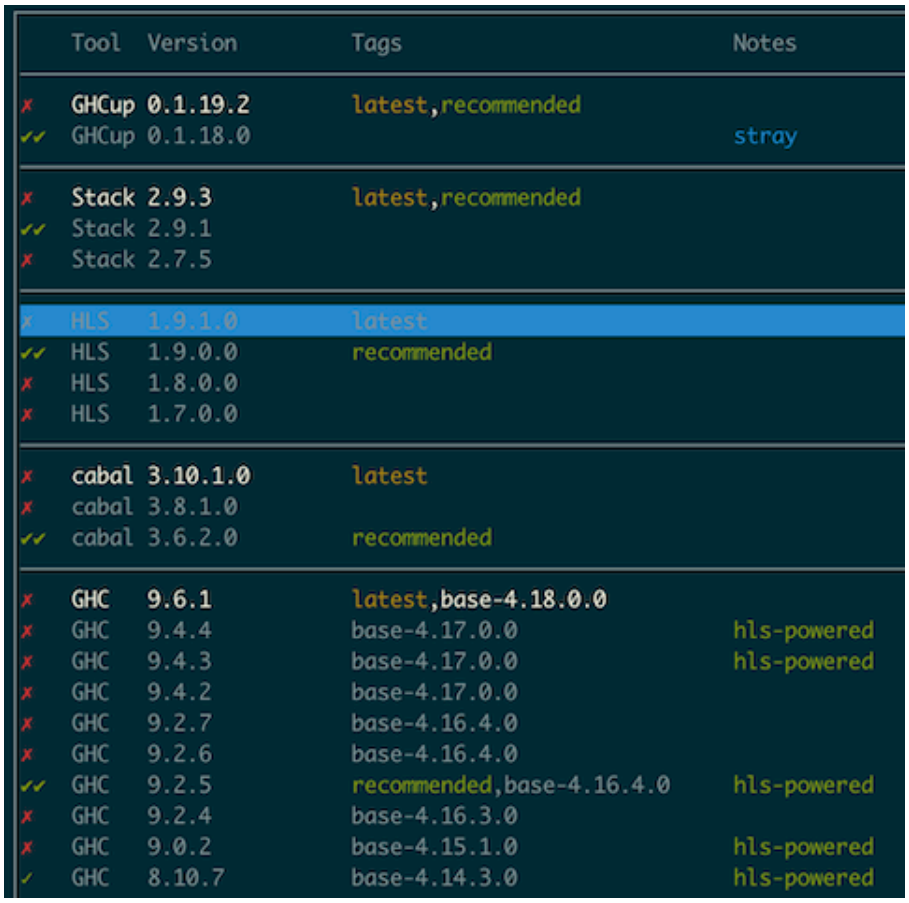
```
-- e.g. if the missing file is
--   ~/.stack/snapshots/aarch64-osx/
134e890a5dd4f65737d6ce4f319374a6d863bbc3ea4aed1de64071c4
72c20a89/9.2.5/doc/case-insensitive-1.2.1.0/doc-
index.json
$ echo {} > ~/.stack/snapshots/aarch64-osx/
134e890a5dd4f65737d6ce4f319374a6d863bbc3ea4aed1de64071c4
72c20a89/9.2.5/doc/case-insensitive-1.2.1.0/doc-
index.json
```

Error using M1 mac + Homebrew + GHC version 8.* from ghcup "Warning: Couldn't figure out LLVM version! Make sure you have installed LLVM between [9 and 13]"
ref: https://robertwpearce.com/one-does-not-simply-use-ghcup-on-macos-m1.html

NO SOLUTION YET

Error using M1 mac + Homebrew + system GHC version 8.* from gchup "ld: symbol(s) not found for architecture x86_64"
Ref: https://github.com/commercialhaskell/stack/issues/3487

ghc 8.10.7 installed via ghcup



	Tool	Version	Tags	Notes
✗	GHCup	0.1.19.2	latest,recommended	
✓✓	GHCup	0.1.18.0		stray
✗	Stack	2.9.3	latest,recommended	
✓✓	Stack	2.9.1		
✗	Stack	2.7.5		
✗	HLS	1.9.1.0	latest	
✓✓	HLS	1.9.0.0	recommended	
✗	HLS	1.8.0.0		
✗	HLS	1.7.0.0		
✗	cabal	3.10.1.0	latest	
✗	cabal	3.8.1.0		
✓✓	cabal	3.6.2.0	recommended	
✗	GHC	9.6.1	latest,base-4.18.0.0	
✗	GHC	9.4.4	base-4.17.0.0	hls-powered
✗	GHC	9.4.3	base-4.17.0.0	hls-powered
✗	GHC	9.4.2	base-4.17.0.0	
✗	GHC	9.2.7	base-4.16.4.0	
✗	GHC	9.2.6	base-4.16.4.0	
✓✓	GHC	9.2.5	recommended,base-4.16.4.0	hls-powered
✗	GHC	9.2.4	base-4.16.3.0	
✗	GHC	9.0.2	base-4.15.1.0	hls-powered
✓	GHC	8.10.7	base-4.14.3.0	hls-powered

stack.yaml

```
resolver: lts-18.21
compiler: ghc-8.10.7


# IMPORTANT: can't use ghc-8.10.7 on M1 macos (aarch64)
# Install and use a separate x86_64
# system-ghc: false
# install-ghc: true
# arch: x86_64
system-ghc: true
install-ghc: false
arch: aarch64
```

error

```
yuinishizawa@G0976.local:haskellInDepth [j=0] >> stack
build
SubLibrary dependency is not supported, this will almost
certainly fail
Linking /Users/yuinishizawa/.stack/setup-exe-cache/
aarch64-osx/tmp-Cabal-
simple_mPHDZzAJ_3.2.1.0_ghc-8.10.7 ...
ld: warning: ld: warning: ignoring file /Users/
yuinishizawa/.ghcup/ghc/8.10.7/lib/ghc-8.10.7/
Cabal-3.2.1.0/libHSCabal-3.2.1.0.a, building for macOS-
x86_64 but attempting to link with file built for macOS-
```

```
arm64
ignoring file /Users/yuinishizawa/.ghcup/ghc/8.10.7/lib/
ghc-8.10.7/template-haskell-2.16.0.0/libHStemplate-
haskell-2.16.0.0.a, building for macOS-x86_64 but
attempting to link with file built for macOS-arm64
ld: warning: ignoring file /Users/yuinishizawa/.ghcup/
ghc/8.10.7/lib/ghc-8.10.7/ghc-boot-th-8.10.7/libHSghc-
boot-th-8.10.7.a, building for macOS-x86_64 but
attempting to link with file built for macOS-arm64

  ...
ld: warning: ignoring file /Users/yuinishizawa/.ghcup/
ghc/8.10.7/lib/ghc-8.10.7/pretty-1.1.3.6/
libHSpretty-1.1.3.6.a, building for macOS-x86_64 but
attempting to link with file built for macOS-arm64
ld: warning: ignoring file /Users/yuinishizawa/.stack/
setup-exe-src/setup-shim-mPHDZzAJ.o, building for macOS-
x86_64 but attempting to link with file built for
unknown-arm64
ld: warning: ignoring file /Users/yuinishizawa/.ghcup/
ghc/8.10.7/lib/ghc-8.10.7/base-4.14.3.0/
libHSbase-4.14.3.0.a, building for macOS-x86_64 but
attempting to link with file built for macOS-arm64
Undefined symbols for architecture x86_64:
  "_ZCMain_main_closure", referenced from:
      _main in ghc_2.o

"_base_ControlziExceptionziBase_absentSumFieldError_clos
ure", referenced from:
     -u command line option

"_base_ControlziExceptionziBase_nestedAtomically_closure
", referenced from:
     -u command line option

"_base_ControlziExceptionziBase_nonTermination_closure",
referenced from:
     -u command line option

"_base_GHCziConcziIO_ensureIOManagerIsRunning_closure",
referenced from:
     -u command line option

"_base_GHCziConcziIO_ioManagerCapabilitiesChanged_closur
e", referenced from:
     -u command line option
  "_base_GHCziConcziSignal_runHandlersPtr_closure",
referenced from:
     -u command line option
  "_base_GHCziConcziSync_runSparks_closure", referenced
from:
     -u command line option

  ...
```

```
  "_hs_cmpxchg64", referenced from:
     -u command line option
  "_hs_cmpxchg8", referenced from:
     -u command line option
  "_hs_main", referenced from:
      _main in ghc_2.o
ld: symbol(s) not found for architecture x86_64
clang: error: linker command failed with exit code 1
(use -v to see invocation)
`gcc' failed in phase `Linker'. (Exit code: 1)


--  While building simple Setup.hs (scroll up to its
section to see the error) using:
      /Users/yuinishizawa/.ghcup/ghc/8.10.7/bin/
ghc-8.10.7 -rtsopts -threaded -clear-package-db -global-
package-db -hide-all-packages -package base -main-is
StackSetupShim.mainOverride -package Cabal-3.2.1.0 /
Users/yuinishizawa/.stack/setup-exe-src/setup-
mPHDZzAJ.hs /Users/yuinishizawa/.stack/setup-exe-src/
setup-shim-mPHDZzAJ.hs -o /Users/yuinishizawa/.stack/
setup-exe-cache/aarch64-osx/tmp-Cabal-
simple_mPHDZzAJ_3.2.1.0_ghc-8.10.7
    Process exited with code: ExitFailure 1
```

NO SOLUTION YET

Fail to profile with error "RTS options are disabled. Link with -rtsopts to enable them."
Problem: need to enable **rtsopts** flag and **clean** the build

e.g.

```
executables:
  main:
    main:               ./app/main.hs
    ghc-options:
     - -rtsopts=all
```

```
# Step 1: IMPORTANT - must clean build
# Otherwise, may not be compiled correctly.
$ stack clean

# Step 2: recompile + run
$ stack --profile build
$ stack --profile exec -- myExe +RTS -p
```

Profiling: Fail to profile with error= "Perhaps you haven't installed the "p_dyn" libraries ... "
Ref: https://gist.github.com/danidiaz/4345c9d93ef7b64368bc7fdee72c5252
Ref: https://github.com/haskell/cabal/issues/2827

Ref: https://stackoverflow.com/questions/32123475/profiling-builds-with-stack

Problem: **-prof flag** is not supported in Cabal
e.g. if package.yaml

```
# package.yaml

...

ghc-options:
- -prof
```

then build fails

```
  $ stack build
  Failed to load interface for 'GHC.Num.BigNat' Perhaps
you haven't installed the "p_dyn" libraries for package
'ghc-bignum'
```

Solution: use the --profile flag instead

```
# Build with profiler
$ stack --profile build

# Run with profiler
$ stack --profile exec -- myMain +RTS -xc
```